

HOMEWORK 5

Instructions

- Please provide the code snippet for each question and sub questions along with its intended output.

Ensure that the code and corresponding output screenshots are placed together, one below the other.

- Work is submitted through your GitHub repository data260-<SID4> (the same repository from HW1, extended further).
- Submit report.pdf (name your submitted PDF as {last_name}_HW5.pdf) with your GitHub repository link included at the top of the report, and confirm the link is working and that both collaborators (Sbnikitha, supriyaselvangesan) have access.
- Keep all application code in shared root-level folders and continue extending your existing codebase from previous homeworks. Do not create a separate copy of the code for this homework.

Section 0 - Personal Configuration and Domain

Before starting, compute your personal configuration values from your SJSU Student ID and record them at the top of your report. These values stay fixed for the rest of the semester.

Value	Definition
SID4	Last four digits of your SJSU Student ID
PORT_BASE	$8000 + (\text{SID4} \bmod 900)$ - the port your app must run on
PREFIX	"s" + SID4
SEED	SID4
VERIFY_SEED	$260000 + \text{SID4}$
DOMAIN_ID	$\text{SID4} \bmod 8$ - determines your assigned domain

At the top of your report, also state your hardware, local model used, and the tagged commit hash for this submission, in addition to the six configuration values above.

Repository Instruction

In your repository, create reports/hw05/ containing:

- RUN_LOG.txt - real console output and timestamps for the runs that produced your reported results.
- raw/ - the machine-readable CSV/JSON files from your experiments (all 150 fault-injection call records: 3 failure rates $\times$ 50 calls each; plus your MCP Inspector test outputs for all tools; plus your agent_runs.jsonl-style log from Part 5).
- METRICS.md - your filled-in results tables.

- `AI_USE.md` - answers to four questions: (1) what you used an AI assistant for and what you did yourself; (2) one AI-produced output that was wrong/unsuitable, or one thing you independently verified; (3) how you detected the problem or verified the result; (4) what you changed and why it works now.
- `report.pdf` - your write-up with screenshots and answers.

`verification.json` - output of a self-check script (e.g. `make verify-hw05`). Write a small script that runs a smoke test on your tagged commit and writes this file. A smoke test just means: start up your system and check that the basic things work, not every edge case. For example: does the FastAPI backend respond on `PORT_BASE`? Do both MCP servers start and respond to a tool call? Your checks should be objective - testing behavior, not exact wording. It should contain:

- Homework number
- `SID4`
- Commit hash
- Model/configuration used (when applicable)
- `SEED / VERIFY_SEED`
- The checks that were run
- Pass/fail for each check

Your script can create temporary test files while it runs, but it must not modify your actual application code.

Tag your submitted repository state `hw5`.

Part 1

This homework builds on HW4. Your existing FastAPI + MySQL backend gets extended with a second entity type that has a meaningful relationship to your primary domain entity - keep extending your existing service, don't start a new one.

I. Database

- Create a second table for a related entity type (e.g., for a book/author-style setup, this second entity plays the role of the "author"). It should include `id` (primary key), a primary text field, a secondary text field, a unique field (e.g., email or code), and timestamps.
- Your primary domain entity's table should contain `id` (primary key), primary field, a unique field (e.g., an ID/code), a numeric field with a sensible default (e.g., an available-count style field), a foreign key to the related entity, and timestamps. Use appropriate data types and constraints.
- Do not store plain-text passwords. Prevent deletion of a related-entity record that still has associated primary-entity records, unless you deliberately implement and document cascade behavior.

II. API Implementation

- Implement CRUD endpoints for both entities.
- For the related entity: add new records, retrieve all with pagination, get individual records, update, delete.
- For your primary domain entity: same set - create, list, retrieve, update, delete.

- Include one additional endpoint to get all primary-entity records associated with a specific related-entity record (the relationship query).
- Use Pydantic schemas for request/response validation. Implement proper HTTP status codes and error messages for resource-not-found, validation errors, and constraint violations. Ensure the unique field's format is validated.
- **Submit Postman screenshots of each action tested using the API endpoints. Submit a screenshot of your DB.**

III. Redux Client

Move your React data layer to Redux Toolkit, integrating with the FastAPI + MySQL backend above.

1. Redux Setup

- Create a Redux store using Redux Toolkit.
- Define a slice to manage the state of your primary domain entity.
- Include reducers/actions for fetching, adding, updating, and deleting records.

2. Async Thunks with Axios

- Implement async thunks that call your FastAPI endpoints: fetch (GET list), create (POST), update (PUT /{id}), delete (DELETE /{id}).
- Handle success and error cases properly.

3. Home Screen

- Display all records by reading from Redux state; the list updates automatically when records are created, updated, or deleted.

4. Create / Update Screens

- Build a form to create a new record and a form to update an existing record (select by ID).
- On submission, dispatch the correct Redux thunk and show the updated results on the Home Screen.

5. Delete Functionality

- Add a delete button next to each record in the list. On click, dispatch the delete thunk and update Redux state to remove the record.

Submit one screenshot per feature (Home, Create, Update, Delete) showing the code snippet (Redux thunk or slice) and the corresponding UI output together, in the same frame. Do not submit full code - only the relevant code snippet and working result.

Part 2 - MCP Tool Servers

A. TheMealDB Tutorial Server

Build a local MCP server in Python that provides four tools for searching for and retrieving recipes from the public TheMealDB API. You will run and debug the server using the MCP Inspector. Connecting it to a desktop MCP client is optional and is not required for credit.

You **do not need to create an API key** for this assignment. TheMealDB allows a test key of 1 for education/dev, and it publishes simple JSON endpoints. (themealdb.com)

What you'll build

A local MCP server named meals that provides four tools:

6. **search_meals_by_name(query, limit=5)**

- Call TheMealDB **Search meal by name** endpoint: `.../search.php?s=<query>`
- Return up to limit meals with: name, area (cuisine), category, thumbnail, and the meal id for drill-down.

7. **meals_by_ingredient(ingredient, limit=12)**

- Call **Filter by main ingredient**: `.../filter.php?i=<ingredient>`
- Return small cards (name, thumbnail, id).

8. **random_meal()**

- Call **Lookup a single random meal**: `.../random.php`.

9. **meal_details(id)**

Call **Lookup a single meal by ID** and return its full recipe details.

The API base is <https://www.themealdb.com/api/json/v1/1/...> and the published "test key" is 1, which is sufficient for this homework.

Set up your environment

10. **Install the MCP Python SDK with CLI**: `pip install "mcp[cli]"`

11. Write a single Python file (e.g., `meals_server.py`) that defines your tools and runs FastMCP over STDIO.

Important logging rule: For STDIO servers, never write to stdout (that corrupts the JSONRPC stream). Log to stderr using a logger.

I/O - Input → Output shapes you should implement:

12. `search_meals_by_name(query, limit)`

- **Input**: query: str, limit: int (1–25)
- **Output** (list): objects with { id, name, area, category, thumb }

13. `meals_by_ingredient(ingredient, limit)`

- **Input**: ingredient: str, limit: int
- **Output** (list): objects with { id, name, thumb }

14. `meal_details(id)`

- **Input**: id: str | int
- **Output** (object): { id, name, category, area, instructions, image, source, youtube, ingredients: [{name, measure}] }

15. `random_meal()`

- **Input**: none
- **Output**: same shape as `meal_details`.
- **Error handling**:
if an endpoint returns "meals": null (no results), return a **clear, empty result** with a short message (e.g., "no matches"). Network or JSON errors should raise a clean error to the client (MCP Inspector will display it).

Run and debug locally with MCP Inspector

16. In your project folder, open a terminal and start development mode with the server file you wrote: `mcp dev <your_file_name.py>`
17. The Inspector opens in your browser. Use it to call each tool with test inputs (e.g., "Arrabiata", ingredient "chicken"), verify outputs look like your designed schema, and fix any errors before moving on.

Include one labeled MCP Inspector screenshot for each of the four tools, showing the input and returned output.

B. Domain MCP Server (required)

- Build a second MCP server over your own <PREFIX>_rel domain data with exactly three tools: search, detail lookup, and one aggregate operation.
- Use one response envelope for all three tools: {ok, data, error}, where error is null on success. This is the same envelope shape you will reuse for `execute_tool` in Part 4 - define it once here and carry it through.
- All MCP STDIO logs go to stderr, never stdout.
- **Validate all three domain tools in MCP Inspector with at least one successful call and one intentionally invalid call per tool. Include screenshots of both - these rejected-call examples are the ones you'll reference again in Part 3.**

Part 3 - Tool Contracts Under Stress

18. Using the same three domain tools and the same rejected-call examples you captured in Part 2B, **Write down expected JSON input schema, the rejected JSON input, and the returned error output for each call. Document why each Part 2B call was rejected** - you do not need to re-run new rejected calls here.
19. Add timeouts and a bounded exponential-backoff retry policy to API or storage operations that may fail. **Demonstrate (Show on report):**
 - **Success on the first attempt.**
 - **Failure on the first attempt followed by success after a retry.**
 - **Failure after all allowed retries, returning a clean error result instead of crashing.**
20. Use `VERIFY_SEED` to reproducibly simulate failures at rates of 0%, 20%, and 50%. The same seed must produce the same success/failure sequence each time. Run 50 calls at each failure rate, for 150 calls total. **For each rate, report the success rate, average latency, and p99 latency.** Save the raw data for all 150 calls in `reports/hw05/raw/`, so your results can be double-checked.
21. Use your results table below to evaluate whether the retry policy is suitable for an interactive assistant. Then **explain how you would modify the number of retries, timeout, or backoff delays for batch processing, where longer processing times are acceptable.**

Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)
0%			
20%			
50%			

Part 4 - One Safe Tool Entry Point and Offline Tests

22. Create `execute_tool(name, inputs)`. This is the single function that the Part 5 agent will use to call any of the three domain tools. It must reuse the `{ok, data, error}` format from Part 2B, return the result as a JSON string, and handle errors without crashing.
23. Write a simple Python test runner using assert statements. For each of the three domain tools, test at least one valid input and the invalid input cases described in Part 3. Include at least six tests overall. Print PASS or FAIL for each test and a final X/Y summary.
24. Use temporary fixtures or dependency injection so the tests run offline, repeatedly, and without an LLM.

Include one screenshot of the terminal running the Part 4 test runner. The screenshot must show the PASS/FAIL result for each test and the final X/Y summary.

Part 5 - Agent Loop, Safety Rule, and Reflection

This part adds a business-logic safety rule and connects the tested tool layer to an Ollama-based agent.

I. Safety rule

- Add one domain-specific safety rule inside `execute_tool`.
- When the rule is violated, return `{ok: false, data: null, error: "..."} without raising an exception.`
- **Demonstrate one allowed call and one blocked call.**

II. Agent loop

- Implement `run_agent(user_input)` using the local Ollama model and the three domain tools.
- Add a turn counter and `max_steps` limit.
- Log each step, tool call, input, result, and final stop reason to `agent_runs.jsonl`.

III. Additional tests

After implementing the Part 5 safety rule and agent loop, add two tests to the existing Part 4 offline test suite. Test that `execute_tool` blocks a call that violates the safety rule, and that `run_agent`, using `MockModel`, stops after reaching `max_steps`. Both tests must run offline without an API key, live model, external API, or database.

IV. Metrics

- Run at least four scenarios using the local Ollama model.
- **Record each scenario's step count, stop reason, and tool-call count in METRICS.md and in report too.**
- A hosted-model comparison is optional.

V. Reflection

Write a 200–300-word reflection in REFLECTION.md and report too:

pick one run from your agent_runs.jsonl-style log and explain, step by step, what the harness did and why it stopped (normal completion, safety-rule block, or max_steps ceiling).

Keep your report concise, organized, and readable. Use clear, short headings and include only the required evidence as mentioned in each parts. Do not submit unnecessary screenshots, full source files, or irrelevant output. Use only labeled screenshots, short relevant code snippets, summary tables, and brief analysis. Label each parts clearly. Ensure your name is clearly visible in every terminal screenshot. If your name is not visible, points may be deducted. Include AIUSE in the report too.

Use consistent formatting throughout the report:

- **Main headings: 14 pt, bold**
- **Subheadings: 12 pt, bold**
- **Body text: 11 pt**

Keep the same sizes and styles throughout the report.